Deadlock

Adapted from EECS 482 at Michigan

Slides by: Harsha V. Madhyastha

Administrivia

Quiz Wednesday

HW5 tests due today

HW5 due Wednesday

HW6 - Cuckoo hashmap out today

- Concurrent data structure
- Deadlocks

Next in class: 2 weeks of related Concurrency ideas

Deadlock

- Synchronization is about constraining executions
- Deadlock is what happens when an execution is over-constrained
  - A must happen before B, B must happen before A
- Example: Swapping classes
  - Alice is in 451, Bob is in 334, and they want to switch
    - » wait for spot to open
    - » add new class
    - » drop old class

Deadlock

- Resources
  - Things needed by a thread that it waits for
  - Examples: locks, disk space, memory, CPU
- Deadlock
  - Cyclical waiting for resources which prevents progress
  - Results in starvation: threads wait forever

BANCO REAL
11455

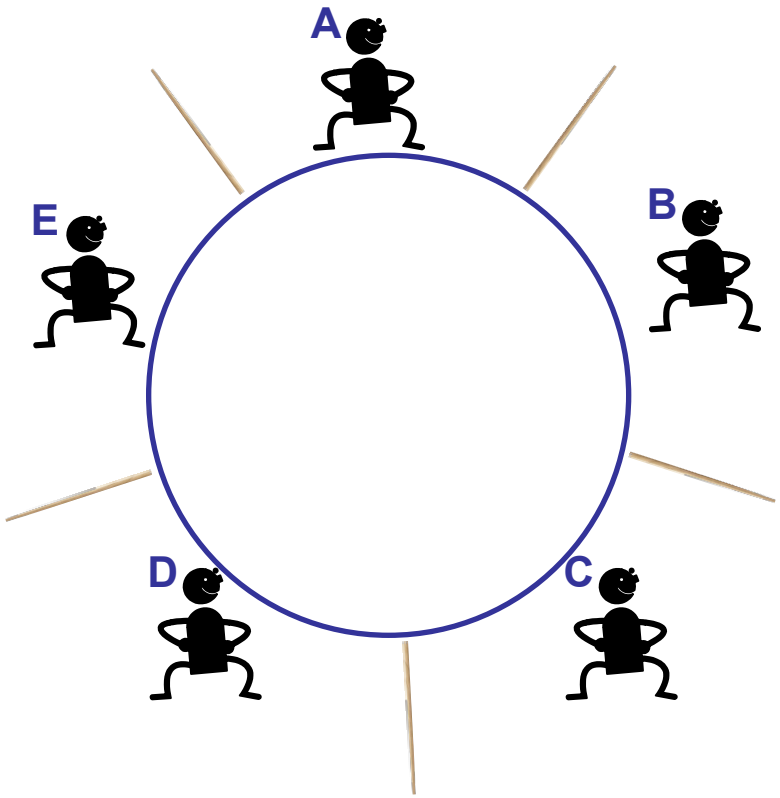
Deadlock example

Thread A	Thread B
x.lock	y.lock
y.lock	x.lock
...	...
y.unlock	x.unlock
x.unlock	y.unlock

- Will a deadlock always occur?

Dining philosophers

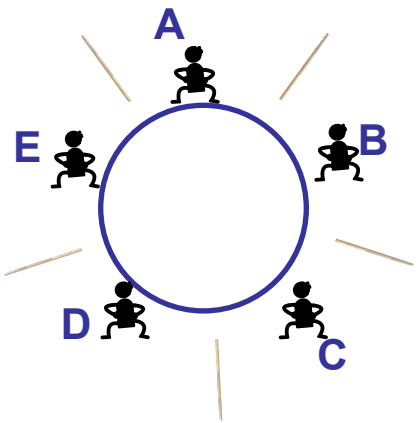
- 5 philosophers sit at round table
- 1 chopstick between each pair of philosophers
- Each philosopher needs 2 chopsticks to eat



Dining philosophers

- Algorithm for philosopher:

```
wait for chopstick on right to be free
pick up chopstick on right
wait for chopstick on left to be free
pick up chopstick on left
put both chopsticks down
```



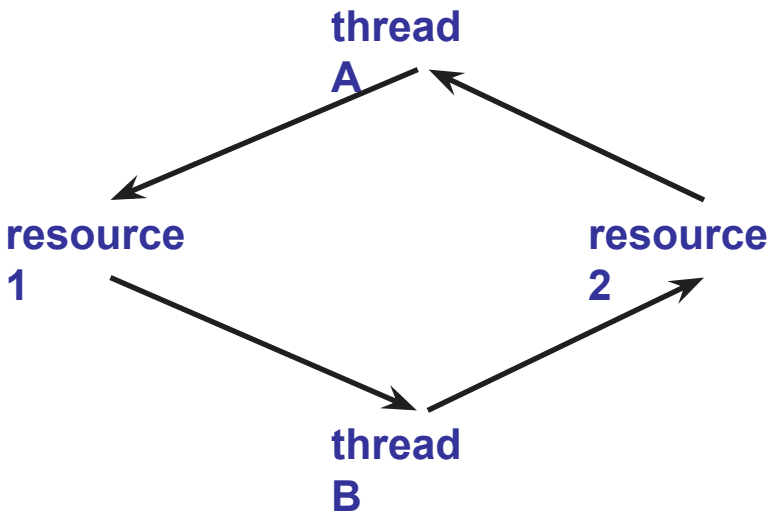
- Can this deadlock?

Generic example of multi-threaded program

```
phase 1:
while (!done) {
   acquire some resource
   work
}

phase 2:
release all resources
```

Waits-for graph



- Cycle represents a deadlock

Strategies for handling deadlock

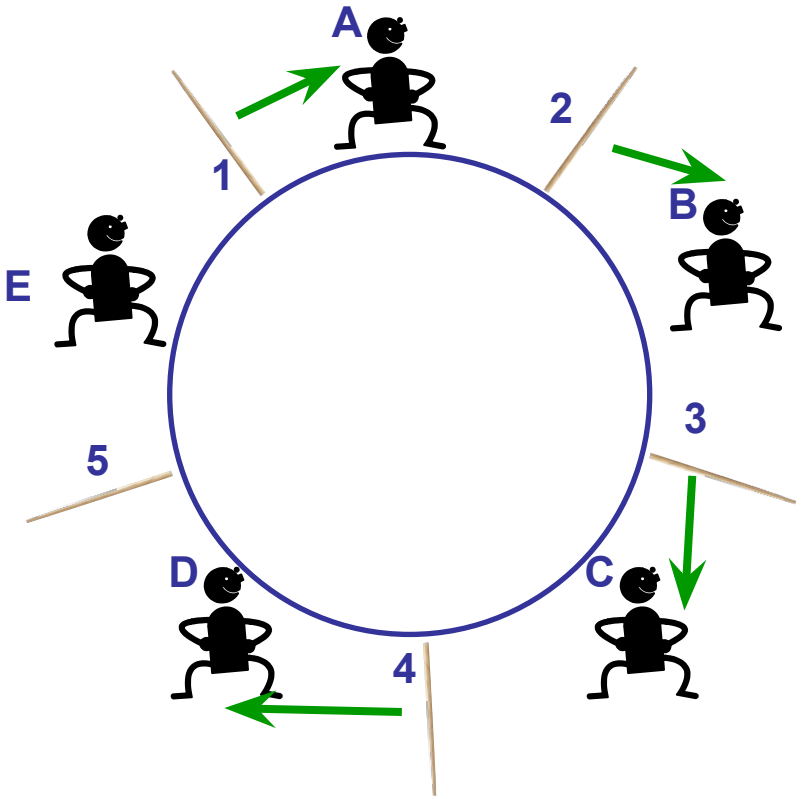
- Ignore
- Detect and fix
  - Detect cycles in the wait-for graph
  - How to fix once detected?
    - » Kill thread, grab resources
    - » Roll back execution
- Prevent

Four necessary conditions for deadlock

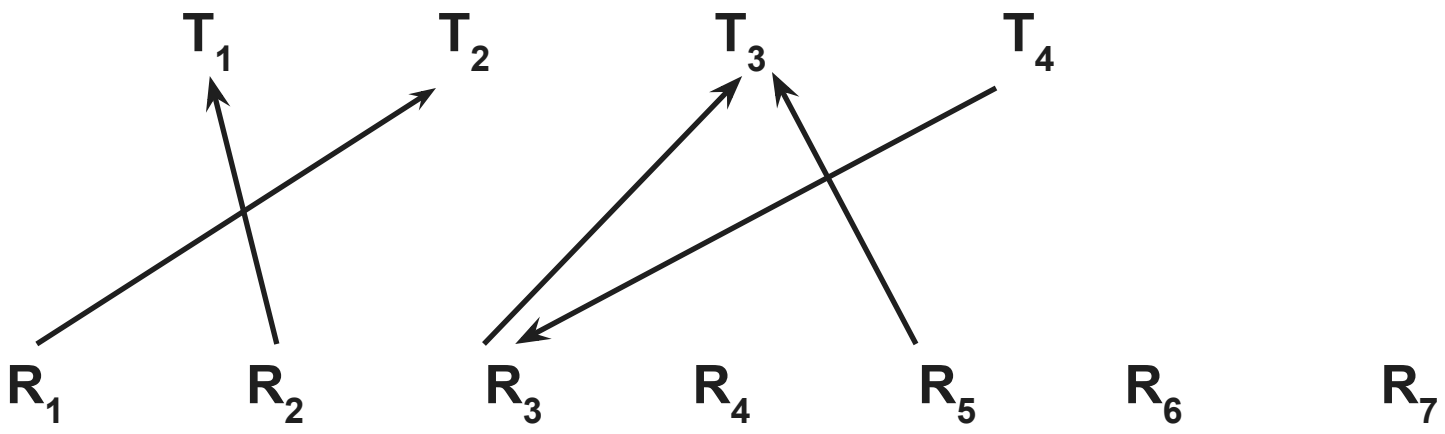
- Limited resources
  - Not enough to serve all threads simultaneously
- No preemption
  - Can’t force threads to give up resources
- Hold and wait
  - Threads hold resources while waiting to acquire other resources
- Cyclical chain of requests

Eliminating circular chain

- Impose global ordering of resources
  - (e.g., acquire chopstick with the smaller number)



Global ordering of resources



Eliminating hold-and-wait

- Two ways to avoid hold and wait:
  - Wait for all resources needed to be free; grab them all atomically
  - If cannot get a resource, release all and start over

- Move resource acquisition to beginning

```
Phase 1a: acquire all resources
Phase 1b: while (!done) {
          work
           }
Phase 2: release all resources
```

Banker's algorithm

- An alternative solution to eliminate hold-and-wait
  - Allows for more concurrency
- Declare resources at the beginning, but don't actually acquire them

```
Phase 1: declare all resources
Phase 2: while (!done) {
         acquire resource if safe
            work
               }
Phase 3: release all resources
```

Banker's algorithm

```
Phase 1: declare all resources
Phase 2: while (!done) {
         acquire resource if safe
            work
               }
Phase 3: release all resources
```

- Only grant resource if it's "safe", otherwise block
  - Safe means I can guarantee that all threads can finish
- Criterion: Can I grant the maximum resources of all threads in some sequential order?

The bank example

- A bank has $6000
- Customers establish credit limit, and then can borrow money (up to their credit limit)
  - Credit limit is “max resource usage”
- When their business is done, customers return the money

Bank solution #1

- *Bank gives money upon request, if it's available*

- Example:
  - Ann asks for credit of $2000
  - Bob asks for credit of $4000
  - Charlie asks for credit of $6000
- Can the bank approve all these credit lines?
- No, a deadlock can form:
  - Ann borrows $1000
  - Bob borrows $2000
  - Charlie borrows $3000
  - Anyone tries to borrow more

Bank solution #1

- *Bank gives money upon request, if it's available*

- Only works if the bank reserves the money when credit line is established. Customers may have to wait at the credit approval stage.
- Example:
  - Ann asks for credit of $2000. Approved.
  - Bob asks for credit of $4000. Approved.
  - Charlie asks for credit of $6000. Must wait until Ann and Bob drop their lines of credit.

Banker's algorithm

- *Bank approves **all** credit requests. Bank gives money upon request but **only if it's safe**.*
- Example:
  - Ann borrows $1000 (bank has $5000 left)
  - Bob borrows $2000 (bank has $3000 left)
  - Charlie wants to take out $2000. Is this allowed?
- It is allowed **iff** there is some sequential ordering of fulfilling everyone's max resources
  - Charlie gets $2000, leaves bank with $1000.
  - Ann can borrow that and finish, "releasing" $2000 back to the bank (total: $2k)
  - So Bob can borrow that and finish, leaving $4000
  - So Charlie can finish.

Limits

Ann:	**$2000**
Bob:	**$4000**
Charlie:	**$6000**

Banker's algorithm

- *Bank approves **all** credit requests. Bank gives money upon request but **only if it's safe**.*
- Example #2:
  - Ann borrows $1000 (bank has $5000 left)
  - Bob borrows $2000 (bank has $3000 left)
  - Charlie wants to take out $2500. Is this allowed?
- It is allowed **iff** there is some sequential ordering of fulfilling everyone's max resources

Limits	
Ann:	$2000
Bob:	$4000
Charlie:	$6000

$500 left in bank. Can't guarantee that any single customer would finish.

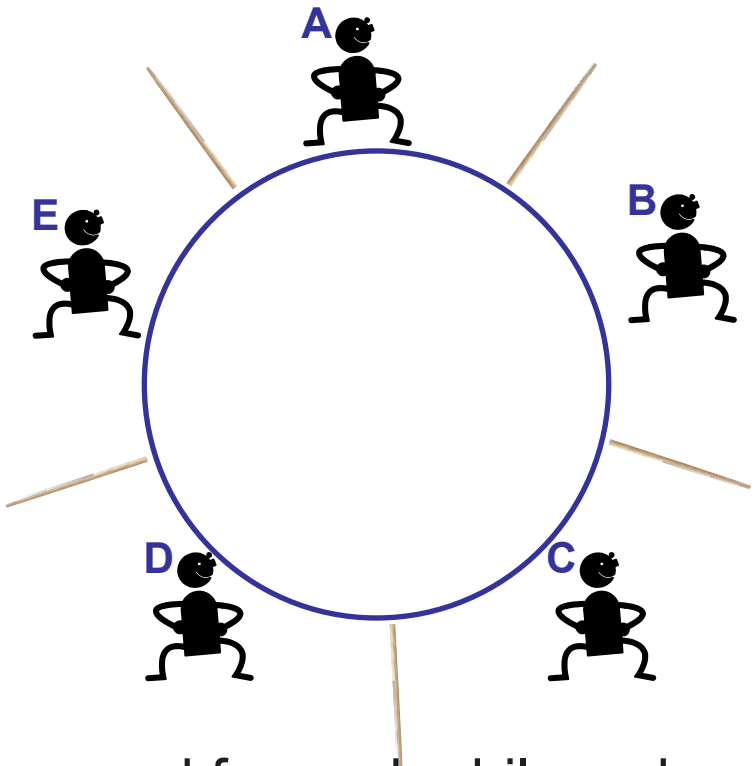
Banker's algorithm

- *Bank approves **all** credit requests. Bank gives money upon request but **only if it's safe**.*
- Example #3:
  - Ann borrows $1000 (bank has $5000 left)
  - Bob borrows $2000 (bank has $3000 left)
  - Charlie wants to take out $2000. Is this allowed?

Limits
Ann: $2000
Bob: $5000
Charlie: $6000

- $1000 left in bank.
  - Ann can finish, leaving $2000 in bank
  - Bob might need $3000 more
  - Charlie might need $4000 more
  - Can't guarantee that either can finish

Banker's algorithm for the dining philosophers



- Max resource need for each philosopher is 2
- Grant chopstick unless it's the last chopstick and nobody has 2

Banker's algorithm

- Allows system to overcommit resources
  - Sum of max resources can be greater than total resources

- Elegant algorithm, when applicable
  - Sometimes it's hard to know what the max resource need is
  - Only applies to "quantitative" resources
    - » Not applicable to locks

Lessons Learned

- Order your locks to prevent cycles
- Declare and acquire resources before doing work to avoid hold-and-wait

Optional Reading

https://docs.rs/ordered-locks/latest/ordered_locks/

https://docs.rs/lock_ordering/latest/lock_ordering/